Personal Portfolio Web App

How this portfolio was built — and how to rebuild it step by step

A practical, CRA-based walkthrough for the EMC Full Stack Final Assignment

Prepared for: Sakthi Prabhu R · September 2026

Contents

§	Section	What you'll learn
1	The Plan	Concept, phases, timeline, architecture decisions
2	Setup — Project Skeleton	CRA app, folder structure, config files
3	Design Tokens & Global Styles	CSS custom properties, light/dark theming
4	Core Wiring	Entry point, App shell, routing, theme context
5	The Data Layer	Single source of truth: profile, skills, projects
6	Hooks & Shared Components	Scroll reveal, buttons, badges, layout pieces
7	Feature Components	Navbar, Hero, cards, skill bars, contact form
8	The Pages	All six routes, one by one
9	Assets & Screenshots	Where project images and avatar go
10	Testing Checklist	What was verified and how to re-verify
11	Deployment	Vercel + Formspree go-live steps
12	Key Takeaways	Patterns worth reusing in any React project

*This guide documents the actual code in **portfolio-frontend/**. Every file path, class name, and snippet below matches the working project shipped alongside this PDF.*

1. The Plan

The goal: a multi-route React portfolio that looks modern, animates with restraint, works on every screen size, and can be extended with a MERN backend later — without rewriting the UI.

1.1 Concept decisions

Decision	Choice	Why
Build tool	Create React App	Zero config, matches the assignment brief
Routing	react-router-dom v6	Standard; nested + dynamic routes
Styling	Plain CSS + custom properties	No build surprises; tokens make theming trivial
Animations	Intersection Observer (no lib)	Scroll reveals with zero bundle cost
Icons	react-icons (Feather/Heroicons)	Tree-shaken; only imported icons ship
Content	JS data modules in src/data/	Separates content from components — easy to update
Backend	Deferred / optional	UI reads local data; swap to fetch later, UI unchanged

1.2 Development phases

Phase	Focus	Output
1. Setup	package.json, public/, gitignore, README	Runnable skeleton
2. Tokens	variables.css + global.css	Design system foundation
3. Core wiring	index.js, App.jsx, ThemeContext	Navigation + theming working
4. Data layer	profile, skills, projects, timeline modules	Content decoupled from UI
5. Shared components	hooks, Button, Badge, SectionReveal…	Reusable building blocks
6. Feature components	Navbar, Hero, ProjectCard, SkillBar, form	Page meat
7. Pages	six routes assembled from components	Full site navigable
8. Assets & polish	resume.pdf, screenshot folders, favicon	Submission-ready
9. Test & verify	build, routes, form, theme, mobile	Confidence
10. Deploy	Vercel + Formspree	Live URL

1.3 Architecture

```
Browser
  ■■■ App.jsx  (BrowserRouter)
        ■■■ ThemeProvider  ← data-theme on <html>, persisted to localStorage
        ■■■ Navbar + Footer  (persistent layout)
        ■■■ <Routes>
```

```
/ ................ Home       (Hero + Featured + About preview)
/about ........... About      (bio + skill bars + timeline)
/projects ........ Projects   (filterable grid)
/projects/:id .... Case study (problem → approach → result)
/contact ......... Contact    (form + socials + resume)
* ................ 404
```

```
Data flow:  src/data/*.js  →  components  →  pages
(Optional MERN later: replace the data import with axios; UI stays identical.)
```

Key insight: content lives in plain JS modules, not inside components. That single decision makes the site maintainable, lets screenshots drop in without code changes, and keeps the door open for a backend swap.

2. Setup — Project Skeleton

Two ways to start: scaffold with CRA and delete the demo, or copy the structure directly.

```
npx create-react-app portfolio-frontend
cd portfolio-frontend
npm install react-router-dom react-icons
npm start   # runs on http://localhost:3000
```

Final folder structure:

```
portfolio-frontend/
■■■ public/
■   ■■■ index.html              # fonts (Space Grotesk + Inter), meta, title
■   ■■■ resume.pdf              # downloadable resume
■   ■■■ assets/
■       ■■■ projects/           # screenshot folders per project id
■■■ src/
■   ■■■ index.js                # entry: imports styles, renders <App/>
■   ■■■ App.jsx                 # router + layout + all routes
■   ■■■ context/ThemeContext.jsx
■   ■■■ hooks/useScrollReveal.js
■   ■■■ data/                   # profile, skills, projects, timeline
■   ■■■ components/
■   ■   ■■■ common/             # Button, Badge, Loader, SectionReveal…
■   ■   ■■■ layout/             # Navbar, Footer, ThemeToggle
■   ■   ■■■ home/               # Hero, FeaturedProjects
■   ■   ■■■ about/              # SkillBar, TimelineItem
■   ■   ■■■ projects/           # ProjectCard
■   ■   ■■■ contact/            # ContactForm, SocialLinks
■   ■■■ pages/                  # Home, About, Projects, ProjectDetail,
■   ■                           # Contact, NotFound
■   ■■■ styles/                 # variables, global, components, animations
■■■ .gitignore  .prettierrc  README.md
■■■ package.json
```

Delete CRA's demo files (App.css, logo, service worker registration). Keep index.html but replace its content with the fonts + meta below.

2.1 index.html essentials

```
<link rel="preconnect" href="https://fonts.googleapis.com" />
<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin />
<link href="https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600&family=Space+Grotesk:wght@500;6
<title>Sakthi Prabhu R — MERN Stack Developer</title>
```

3. Design Tokens & Global Styles

Define tokens BEFORE component styles — the guide's step 3, and the reason re-theming later is a five-line change instead of a rewrite.

```
:root {
  --font-display: "Space Grotesk", sans-serif;
  --font-body: "Inter", sans-serif;
  --bg: #f7f8fa;   --bg-elevated: #ffffff;
  --text: #16181d; --text-muted: #5a6070;
  --accent: #4f46e5;  --accent-soft: rgba(79,70,229,.1);
  --border: #e3e6ec;
  --radius: 10px;  --radius-lg: 16px;
  --dur: 300ms;    --ease: cubic-bezier(.22,1,.36,1);
}
[data-theme="dark"] {
```

```
  --bg: #0b0c10;  --bg-elevated: #15161c;
  --text: #f2f3f5; --text-muted: #9aa0ae;
  --accent: #818cf8;  --border: #262833;
}
```

Global styles (src/styles/global.css) — resets, typography scale, focus states:

```
body { font-family: var(--font-body); background: var(--bg); color: var(--text); }
h1,h2,h3 { font-family: var(--font-display); }
:focus-visible { outline: 3px solid var(--accent); outline-offset: 2px; }
.container { max-width: var(--container-max); margin: 0 auto; padding: 0 1.5rem; }
.section { padding: var(--space-7) 0; }
```

Four stylesheets keep things findable: variables.css (tokens), global.css (element styles), components.css (all component classes), animations.css (reveal classes).

4. Core Wiring

4.1 Entry point — src/index.js

```
import React from "react";
import ReactDOM from "react-dom/client";
import App from "./App";
import "./styles/variables.css";
import "./styles/global.css";
import "./styles/components.css";
import "./styles/animations.css";

ReactDOM.createRoot(document.getElementById("root")).render(
  <React.StrictMode><App /></React.StrictMode>
);
```

4.2 App shell — src/App.jsx

```
<ThemeProvider>
  <BrowserRouter>
    <ScrollToTop />          {/* scroll resets on route change */}
    <Navbar />               {/* persistent */}
    <main id="main-content">
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
        <Route path="/projects" element={<Projects />} />
        <Route path="/projects/:id" element={<ProjectDetail />} />
        <Route path="/contact" element={<Contact />} />
        <Route path="*" element={<NotFound />} />
      </Routes>
    </main>
    <Footer />
  </BrowserRouter>
</ThemeProvider>
```

4.3 Theme system — src/context/ThemeContext.jsx

Three concerns, one small file: read stored theme (fallback: OS preference), apply it as a data-theme attribute on <html>, persist every toggle.

```
const getInitialTheme = () => {
  const stored = localStorage.getItem("theme");         // 1. saved choice
  if (stored === "light" || stored === "dark") return stored;
  return matchMedia("(prefers-color-scheme: dark)").matches ? "dark" : "light";
};
// Provider: on every theme change →
document.documentElement.setAttribute("data-theme", theme);  // 2. apply
localStorage.setItem("theme", theme);                        // 3. persist
```

Because all colors are CSS variables, the entire dark mode is just the [data-theme] selector swapping variable values — no component changes anywhere.

5. The Data Layer

Four modules in src/data/ hold ALL content. Components never hardcode text — they import.

File	Exports	Consumed by
profile.js	name, role, tagline, bio[], socials, resumePath	Navbar, Hero, Contact, Footer

File	Exports	Consumed by
skills.js	skillGroups[] with name + level	About (SkillBar)
projects.js	projects[] with full case study, filter tags, image paths	Home, Projects, ProjectDetail
timeline.js	timeline[] (internships, education), certifications[]	About

The project shape — one object powers card AND detail page:

```
{
  id: "betbillsbot",              // → /projects/betbillsbot, folder name for screenshots
  title: "BetBillsBot",  subtitle: "Node.js Telegram Bot",
  tags: ["Node.js", "Telegram"],  filterTags: ["Node.js"],   // card chips / filterable
  image: "/assets/projects/betbillsbot/cover.png",            // drop a file here = done
  gallery: [],  liveUrl: "",  repoUrl: "",  featured: true,
  short: "One-liner for the card.",
  problem: "",  approach: "",  result: "",    // the case study
  highlights: ["…"],  stack: ["Node.js", "Railway"]
}
```

6. Hooks & Shared Components

6.1 The scroll-reveal hook — src/hooks/useScrollReveal.js

One IntersectionObserver per element; when 15% becomes visible, flip state once and disconnect. CSS does the rest via a transition on .reveal-visible.

```
export default function useScrollReveal(options = {}) {
  const ref = useRef(null);
  const [visible, setVisible] = useState(false);
  useEffect(() => {
    const node = ref.current;
    if (!node) return;
    const observer = new IntersectionObserver(([entry]) => {
      if (entry.isIntersecting) { setVisible(true); observer.disconnect(); }
    }, { threshold: 0.15, ...options });
    observer.observe(node);
    return () => observer.disconnect();
  }, []);
  return { ref, visible };
}
```

6.2 SectionReveal — the wrapper everything uses

```
<SectionReveal direction="left" delay={120}>
  <ContactForm />
</SectionReveal>
/* renders: <div class="reveal reveal-left reveal-visible"> … */
/* animations.css:
   .reveal           { opacity:0; transform: translateY(24px); transition: … }
   .reveal-visible   { opacity:1; transform: none; }                          */
```

6.3 Other shared pieces

Component	Job
Button	Renders Link (internal) / a (external) / button — one style API
Badge	Tech tag pill used on cards, detail facts, filters
Loader	Spinner + label, role=status for screen readers
ScrollToTop	useEffect on pathname → window.scrollTo(0,0)
SectionHeading	Kicker + title + subtitle block for page tops

7. Feature Components

7.1 Navbar

- Sticky (position:fixed) with backdrop blur; height in --nav-height so pages offset via padding-top
- NavLink with isActive → .active class → animated underline (::after scaleX)
- Hamburger below 768px: menu slides down (transform + opacity transition)
- GitHub/LinkedIn/resume/email icons + ThemeToggle in both desktop and mobile menu

7.2 Hero

- Two-column grid: copy left, avatar right (stacks + centers under 900px)
- Avatar: conic-gradient ring slowly rotating (animation: spin-slow 14s) around initials — swap to a photo by uncommenting one <img>
- Stats row (4+ projects · 2 internships · 1 open-source) pulled from resume facts
- CTAs use shared Button: View my work → /projects, Get in touch → /contact

7.3 ProjectCard

- Thumbnail with onError fallback → branded “screenshot coming soon” panel, so missing images never look broken
- Whole thumb links to the case study; body shows subtitle, one-liner, tag badges
- Live/Code links render only when URLs exist; otherwise “More on GitHub”
- Hover: translateY(-6px) + shadow + accent border — pure CSS, no JS

7.4 SkillBar

```
const { ref, visible } = useScrollReveal({ threshold: 0.4 });
<div className="skill-track" role="progressbar" aria-valuenow={level} …>
  <div className="skill-fill" style={{ width: visible ? `${level}%` : "0%" }} />
</div>
```

Width starts at 0% and transitions to the target when scrolled into view — the bar “fills”. role=progressbar keeps it accessible.

7.5 ContactForm — the most stateful component

State	UI
idle	Clean inputs, Send message button
sending	Inputs disabled, spinner in button, “Sending…”
success	Green banner, fields cleared
error	Red banner with retry hint

```
// validate(): required name/email/message + regex email + min length
// onSubmit: if REACT_APP_FORMSPREE_ID set → POST JSON to formspree
//           else → 900ms simulated delay (demo mode)
setStatus("sending"); … setStatus("success"); setValues({name:"",email:"",message:""});
```

8. The Pages

Page	Assembled from	Notes
Home	Hero + FeaturedProjects + About preview	featuredProjects = filter(featured)
About	SectionHeading + bio + SkillBars + Timeline + certs	Two-column grid, stacks on mobile
Projects	SectionHeading + filter chips + grid	Filter = useState('All') + .filter()
ProjectDetail	Cover + problem/approach/result + facts aside	useParams() → getProjectById; unknown id → friendly not-found
Contact	SocialLinks + resume button + ContactForm	Two-column, stacks on mobile
NotFound	Gradient 404 + Back home	path="*" catches everything

Filtering — deliberately library-free:

```
const [filter, setFilter] = useState("All");
const visible = filter === "All"
  ? projects
  : projects.filter(p => p.filterTags.includes(filter));
// chips: aria-pressed={filter === tag} for accessibility
```

ProjectDetail reads useParams(), looks up the project, and renders a two-column layout: narrative sections left, facts (stack, links, highlights) in a sticky aside right. “More projects” links the remaining three.

9. Assets & Screenshots

The asset contract is deliberately boring: files in public/ are served at the site root, and data paths already point at the expected locations.

```
portfolio-frontend/public/assets/projects/
  ■■■ dynamic-image-gallery/cover.png
  ■■■ favourite-student-list/cover.png
  ■■■ betbillsbot/cover.png
  ■■■ redstore/cover.png

# that's it — /assets/projects/<id>/cover.png is already the image path
# extras: add 2.png, 3.png … and list them in that project's gallery[]
```

- Cover ratio 16:9, under ~500 KB, capture at 1440×900
- Avatar photo → public/assets/avatar.jpg, then uncomment the <img> in Hero.jsx
- Resume → public/resume.pdf (already linked in navbar + contact)
- Missing files are safe: cards show a branded fallback, never a broken-image icon

10. Testing Checklist (as verified)

Check	How it was verified
Production build compiles	npm run build → Compiled successfully, ~64.6 kB JS gzip
All 6 routes render	Navigated each route in a real browser
Project filtering	Clicked “Node.js” chip → only BetBillsBot shown
Case study data	H1, problem/approach/result, stack badges all populated
Form validation	Empty submit → 3 field errors; bad email → format error
Form success state	Valid submit → sending spinner → success banner, fields cleared
Theme persistence	Toggle → data-theme + localStorage verified; survives reload
404 route	/does-not-exist → styled 404 with Back home
Resume served	GET /resume.pdf → 200, 114,993 bytes
No console errors	Console clean except React Router future-flag notices
Broken-image safety	Cards without screenshots show branded fallback
Mobile layout	Hamburger menu, stacked grids under 768/900px breakpoints

Re-run anytime:

```
npm run build     # must compile clean
npm start         # http://localhost:3000 — walk every route
# plus: tab through the site (keyboard nav), Lighthouse audit in Chrome DevTools
```

11. Deployment

Frontend — Vercel (or Netlify):

1. Push portfolio-frontend/ to a public GitHub repo
2. vercel.com → Add New → Project → import the repo
3. Framework preset: Create React App (build: npm run build, output: build/)
4. Deploy — every future git push auto-deploys

Contact form — Formspree (2 minutes):

1. formspree.io → New form → copy the form ID (the part after /f/)
2. Add env var in Vercel: REACT_APP_FORMSPREE_ID = → redeploy
3. The form now really sends; without the var it stays in demo mode

Optional MERN backend (extra credit):

- Express API with GET /api/projects and POST /api/contact (Mongoose → MongoDB Atlas)
- Deploy to Render as a Web Service; set MONGO_URI + PORT env vars there
- In the frontend, swap the src/data import for an axios call with loading state — keep local data as fallback
- CORS restricted to your Vercel domain; never commit .env

Pre-submission checks:

- Test the LIVE url, not localhost — routes, form, resume, theme
- README with setup instructions and a screenshot
- Lighthouse ≥ 90 performance + accessibility
- .env / secrets not committed

12. Key Takeaways

1. Data modules beat hardcoded content — updating a project is editing one JS object
2. Design tokens first: dark mode became 8 lines of CSS because every color was a variable
3. One animation mechanism (IntersectionObserver + CSS classes) covers reveals, skill bars, and nav states
4. Graceful fallbacks everywhere: missing screenshot → branded panel; missing form ID → demo mode
5. Accessibility is cheap if done early: semantic HTML, focus-visible, aria-pressed, role=progressbar
6. The backend was optional by design — the data-layer boundary means MERN can bolt on later with zero UI rewrites

Built with Create React App · React Router v6 · react-icons — September 2026